Lab 10 – (Week of October 28th)

All of the activities of this lab are to be completed in teams of two (or three, but only as necessary), and each team is limited to the use of one computer.

Activity #1: Refactoring the given `Doubles.java` program to use the given `Die` Class

Consider the given version of the sample program, `Doubles.java`, which was presented in Chapter 5 of the course text. This program uses two integer variables and the `random` method from the `Math` class to simulate the rolling of two dice and counts how many such rolls it takes until "doubles" occurs.

For this activity you are to "refactor" this program so that instead of using two integer variables it makes use to two `Die` objects to perform its task. The resultant program should be clean of any extraneous declarations and statements. There is no need to change the name of the program, and the external behavior of the program is basically the same. As much as possible the output of the program should be like that shown at the right.

```
next roll = [5][2]
next roll = [6][5]
next roll = [5][4]
next roll = [2][2]
Doubles after 4 rolls
```

Although an integer variable can be directly used to hold the value of a die this is a very weak abstraction. Thus, the point of this activity is to have you modify a simple working program so as to make use of a much stronger abstraction. Hopefully this then will give you a better feel for and appreciation of Object-oriented programming.

Once you are convinced that you have the revised `Doubles.java` program working you are to show it to your lab instructor, explain it and demonstrate it to them. Be sure that you have revised and completed the comments at the beginning of this source code file (providing the names of all team members) before you submit it to the CMPS 134L CWS for the section you are working in.

Activity #2: The `Account.java` Instance Class and it's Client, `AccountClient.java`

Consider the given sample source programs, `Account.java` and `AccountClient.java`, which together provide for the simulation of the management of a financial account. More specifically, `Account` is an instance class that presents a description for basic `Account` objects; an abstraction of a financial account. The `AccountClient` application program presents an interactive interface to the user, prompting for, recognizing and processing simple commands pertaining to such accounts.

First, you are to study these source programs then you are to execute the `AccountClient` program enough to gain an understanding of what it does and likewise how it does what it does. The key thing here, of course, is to appreciate the state and behaviors of the `Account` class as it is used by it's client, `AccountClient`.

Second, you are to accomplish each of the following modifications:

- You'll note that all of the "character commands" of the `AccountClient` program are special symbols, except for the letter '`Q`' which is used to "Quit" execution of the program. As a convenience, modify the `AccountClient` program so that both "`Q`" and "`q`" are recognized for this purpose.

- Add a "command" to the `AccountClient` program that prints the "Account's Information" for the user to see. The information I'm referring to is the string returned by the `toString` method of the `Account` Class. Use the character '**#**' to be the command character for this operation.

  The expectation is that you will follow the structure of the program as given and modify it in a clear, straightforward and consistent manner.

- Accounts are typically identified by some sort of "Account Number" that is bound to the account from its creation onward, and are normally not allowed to change. The `Account` class, as given, has no such provision in it. Modify the `Account.java` source program so that each `Account` object is given an account number at the time it is constructed.

  Once this modification is made you'll need to provide a corresponding accessor method to allow clients to "get" the Account Number of such objects. Also, modify the `toString` method so that the object's Account Number is prefixed to the resultant string in some clear fashion; see sample dialogue presented at the end of this activity's specification.

- You'll note that the "=" command in the `AccountClient` program is used to print the current balance of the account. Since the "internal representation" used in the class is as an `int`, the balance is really the balance of the account expressed in pennies. Thus an account balance of one thousand two hundred and thirty four (pennies) prints as `1234` when it really should be printed as `$12.34`.

  Modify the `AccountClient` program to accomplish this "fix". As a hint I'll offer that the "tricky" part will be to assure that the "cents" always has two places with a leading zero as needed. Thus, two hundred and seven should be `$2.07`., and likewise eight cents should be `$0.08`.

- Both the `increase` and `decrease` methods in the `Account` class allow for negative amounts to be passed as actual parameter values. This is potentially problematic, since a negative increase has the effect of a decrease and likewise a negative decrease has the effect of an increase. This is potentially confusing (and maybe even illegal?)

  Thus, you are to modify the implementations of both the `increase` and `decrease` methods to prevent negative actual parameter values from being used to mutate the account balance. It should be clear how to accomplish this. However, the real question is how is the client to know that their call to `increase` or `decrease` actually mutated the object?

  The preferred option is to have these methods `throw` an `IllegalArgumentException` when the actual parameter values are problematic. In this way the client can `catch` the exception and handle it as it sees fit. Thus, it will be necessary to modify `AccountClient.java` accordingly.

  A second, and inferior, option is to make both the `increase` and `decrease` methods functions that return `true` if and only if they were able to complete their tasks and `false` otherwise. In this way the client can test the value returned by these methods to determine whether or not the operation was completed.

  Choose one of these options and then modify both the `Account.java` and `AccountClient.java` source programs accordingly to effect this change.

To assist you in visualizing what the completely modified `AccountClient` application program may present to the user, consider the dialogue given to the right.

Once you are convinced that you have the `Account.java` and `AccountClient.java` source programs revised and working you are to show them to your lab instructor and demonstrate them as well.

Be sure that you have revised and completed the comments at the beginning of these source code files (providing the names of all team members) before you submit them both to the CMPS 134L CWS for the section you are working in.

```
PMJ's AccountClient...
Enter a command:>=
$0.00
Enter a command:>+
Enter Amount:>10000
Operation performed.
Enter a command:>=
$100.00
Enter a command:>-
Enter Amount:>-17
Operation not performed.
Enter a command:>=
$100.00
Enter a command:>#
17: $100.00
Enter a command:>-
Enter Amount:>2500
Operation performed.
Enter a command:>#
17: $75.00
Enter a command:>=
$75.00
Enter a command:>?
Commands are #=+-?Qq
Enter a command:>grisgrop
Try command again:>+
Enter Amount:>1234
Operation performed.
Enter a command:>=
$87.34
Enter a command:>-
Enter Amount:>30
Operation performed.
Enter a command:>#
17: $87.04
Enter a command:>=
$87.04
Enter a command:>q
Done!!!
```